Virtual Interfaces & veth-pair

Panoramica e creazione in C

Roberto Marino, PhD

Università di Messina
CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

April 13, 2026

UniMe
1548

Interfacce fisiche vs. virtuali

Interfaccia fisica

- Collegata a hardware reale (NIC, WiFi. . .)
- Il kernel scrive/legge frame **su cavo/etere**
- Esempio: `eth0`, `wlan0`

Interfaccia virtuale

- Nessun hardware — emulata **in kernel**
- I pacchetti circolano **solo in memoria**
- Il kernel le tratta esattamente come quelle fisiche

Principio chiave

Per il **kernel network stack** non esiste differenza tra interfaccia fisica e virtuale: routing, firewall, socket — tutto funziona allo stesso modo. La differenza è **solo nel driver**: invece di scrivere su un bus PCI, il driver consegna i pacchetti a un file descriptor (TUN/TAP) oppure all'altra estremità di un tubo (veth).

Modello di interfaccia: Ogni interfaccia ha due lati

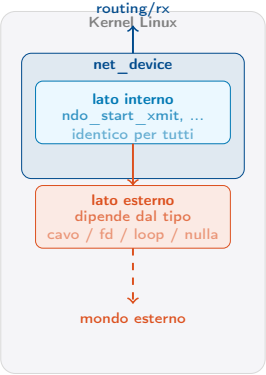
Lato interno — verso il kernel (uguale per tutti)

- `ndo_start_xmit(skb, dev)` — trasmetti un pacchetto
- `ndo_open(dev)` — porta UP
- `ndo_stop(dev)` — porta DOWN
- `ndo_get_stats64(dev,s)` — statistiche

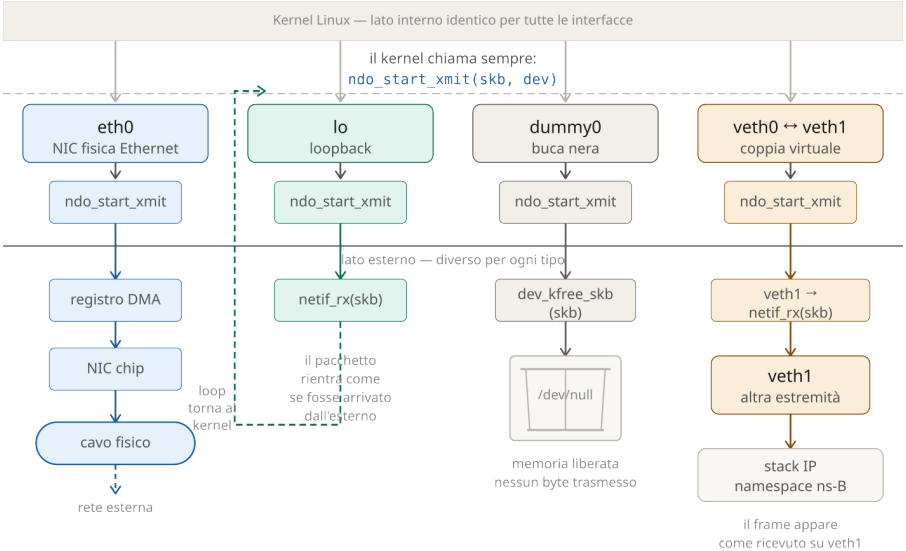
Il kernel non sa nulla di cavi, pipe o file descriptor. Chiama queste funzioni e si fida del driver

Lato esterno — verso il mondo

- `eth0` → DMA → cavo fisico
- `lo` → `netif_rx()` → torna al kernel
- `tun0` → coda → `read(fd)` del processo
- `dummy0`→ `dev_kfree_skb()` → nulla
- `veth0` → `veth1->netif_rx()` → altra iface



Principali interfacce



ndo_start_xmit e netif_rx

ndo_start_xmit — trasmissione

Puntatore a funzione in struct net_device_ops. Il kernel la chiama dopo routing e incapsulamento. Cosa fa dentro: affar del driver.

```
netdev_tx_t (*ndo_start_xmit)(
struct sk_buff *skb,
struct net_device *dev);
// ritorna NETDEV_TX_OK o NETDEV_TX_BUSY
```

netif_rx — ricezione

Funzione del kernel (non del driver). Il driver la chiama quando un pacchetto arriva. Il kernel smista nelle code e risale i protocolli.

Dove sono definite

ndo_start_xmit	include/linux/netdevice.h
netif_rx	net/core/dev.c
lo driver	drivers/net/loopback.c
dummy driver	drivers/net/dummy.c
veth driver	drivers/net/veth.c

sk_buff

La "busta" che viaggia tra le due funzioni: byte del pacchetto, puntatori agli header di ogni livello, interfaccia di ingresso/uscita, metadati. Passa di mano tra driver, routing, protocolli e socket.

Cosa fanno per ogni tipo di interfaccia

Tipo	ndo_start_xmit	netif_rx
`eth0`	Scrive nel DMA della NIC. Il chip trasmette sul cavo.	Chiamata dal driver quando arriva un frame fisico.
`lo`	Chiama direttamente `netif_rx(skb)`: il pacchetto rientra nello stesso stack.	Chiamata da `loopback_xmit()` — trasmissione e ricezione coincidono.
`dummy`	`dev_kfree_skb(skb)`: libera la memoria. Due righe. Fine.	Mai: il kernel risponde via route `local` prima di arrivare al driver.
`veth0`	`veth1->netif_rx(skb)`: inietta il pacchetto nello stack dell'altra estremità.	Chiamata da `veth_xmit()` di `veth1` quando trasmette verso `veth0`.

Pattern comune

`lo` e `veth` usano entrambi `netif_rx` dentro `ndo_start_xmit`. La differenza: `lo` reinietta sullo stesso stack, `veth` lo inietta nell'altra estremità — eventualmente in un namespace separato.

Interfaccia ≠ indirizzo IP

Due oggetti separati nel kernel

- `net_device` — il canale di trasmissione. Esiste senza alcun IP.
- `in_ifaddr` — un indirizzo IP assegnato. Opzionale, zero o molti per interfaccia.

Quando assegni un IP il kernel fa tre cose

1. Crea `in_ifaddr` e lo collega a `net_device`
2. Aggiunge rotta nella tabella `main`: `10.0.0.0/24 dev eth0`
3. Aggiunge rotta nella tabella `local`: `local 10.0.0.1 dev eth0`

Interfaccia senza IP — caso normale

Porta di un bridge: riceve e trasmette frame L2 senza nessun indirizzo di livello 3.

`tcpdump` cattura su qualsiasi interfaccia anche senza IP assegnato.

La tabella `local`

Quando `ping 10.0.0.1` e quell'IP è tuo, il kernel risponde direttamente via route `local` — senza passare per `ndo_start_xmit`.

Per questo `ping` sull'IP di `dummy0` riceve risposta anche se il driver butta via tutto.

Tutte le interfacce virtuali — gruppo 1: kernel-only

`lo` — loopback

- Nasce automaticamente in ogni namespace
- `127.0.0.1/8` e `::1/128`
- `ndo_start_xmit` chiama `netif_rx(skb)`
- Il pacchetto rientra nello stack come se fosse arrivato
- MTU 65536 — nessun cavo da saturare
- **Non si crea via Netlink**

`dummy` — buca nera

- `ndo_start_xmit` chiama `dev_kfree_skb(skb)`
- I pacchetti in uscita vengono scartati
- Nessun traffico in entrata reale
- Utile per IP stabile (VIP), test, Kubernetes
- Creazione: `RTM_NEWLINK + "dummy"` — 60 byte

`nlmon` — monitor Netlink

- Cattura tutto il traffico Netlink del sistema
- Usata con `tcpdump` o `wireshark`
- Wireshark ha il dissector completo per Netlink

`ifb` — Intermediate Functional Block

- Reindirizza traffico per applicare `tc` in ingresso
- Il kernel non supporta `tc qdisc` in ingresso direttamente
- Soluzione: redirigi su `ifb0` e applica `tc` in uscita

Tutte le interfacce virtuali — gruppo 2: coppie e bridge

`veth` — virtual Ethernet pair

- Coppia: `veth0` ↔ `veth1`
- Pipe bidirezionale L2: entra da un lato, esce dall'altro
- `veth0.ndo_start_xmit` chiama `veth1->netif_rx()`
- Frame Ethernet completi, ARP incluso
- Fondamento di Docker, LXC, Kubernetes
- Creazione: `RTM_NEWLINK + "veth"` — 96 byte

`bridge` — switch virtuale L2

- Collega più interfacce come porte di uno switch
- Apprende i MAC e instrada i frame
- Supporta STP/RSTP per loop prevention
- Supporta VLAN filtering per porta
- Usato da libvirt, Docker (modalità bridge)

`vxcan` — virtual CAN pair

- Come `veth` ma per bus CAN (automotive)
- Isola software ECU in container separati
- Usato in architetture SDV (Software Defined Vehicle)

Bridge con due porte da shell

```
sudo ip link add br0 type bridge
sudo ip link add veth0 type veth peer veth1
sudo ip link set veth0 master br0
sudo ip link set veth1 master br0
sudo ip link set br0 up
```

Bridge e IP

Il bridge stesso può avere un IP (interfaccia di gestione). Le porte non ne hanno — operano a L2.

Tutte le interfacce virtuali — gruppo 3: consegna a processo

tun — tunnel L3

- Il processo apre `/dev/net/tun` + `ioctl(TUNSETIFF)`
- `ndo_start_xmit` mette `skb` in una coda
- Il processo legge pacchetti IP grezzi con `read(fd)`
- Il processo scrive pacchetti IP con `write(fd)`
- Usato da: OpenVPN, WireGuard, OpenConnect

tap — tunnel L2

- Come `tun` ma consegna frame Ethernet completi
- Il processo vede header MAC, ARP, VLAN
- Usato da: QEMU/KVM (NIC della VM), VDE
- Può essere aggiunto a un bridge: la VM appare come host fisico nella LAN

TUN: flusso di un pacchetto VPN

Il pacchetto destinato a `10.8.0.x` viene instradato su `tun0`, il processo VPN lo cifra e lo re-invia cifrato su `eth0`.

Tutte le interfacce virtuali — gruppo 4: dipendenti da padre

`vlan` — 802.1Q tagging

- Vive sopra un'interfaccia padre (`eth0.100`)
- Aggiunge/rimuove tag 802.1Q ai frame
- Ogni VLAN ha il proprio stack IP
- Creazione: `ip link add eth0.100 link eth0 type vlan id 100`

`macvlan` — MAC multipli su una NIC

- Più interfacce virtuali, ognuna con MAC diverso
- Modalità: bridge, private, vepa, passthru
- Usato per container che devono apparire sulla LAN fisica con IP propri

`ipvlan` — IP multipli, stesso MAC

- Come macvlan ma condivide il MAC del padre

`bond` — aggregazione NIC

- Più NIC fisiche → un'unica interfaccia logica
- Modalità: active-backup, round-robin, LACP (802.3ad)
- Failover automatico se una NIC cade

`team` — alternativa moderna a bond

- Architettura modulare con `teamd` in userspace
- Più flessibile ma meno diffuso di bond

Tunnel: `gre`, `ipip`, `sit`, `vxlan`

- Incapsulano traffico in altri protocolli
- `vxlan`: L2 in UDP — overlay Kubernetes
- `sit`: IPv6 in IPv4 — transizione
- `gre`: IP in IP — VPN aziendali

Riepilogo — cosa fa il lato esterno di ogni tipo

Tipo	Liv.	ndo_start_xmit fa...	Uso principale
`lo`	L3	`netif_rx(skb)` — loop	IPC locale
`dummy`	L2	`dev_kfree_skb(skb)` — nulla	VIP stabile
`veth`	L2	`peer->netif_rx(skb)`	container/ns
`bridge`	L2	lookup MAC → porta di uscita	switch virtuale
`tun`	L3	coda → `read(fd)` processo	VPN IP
`tap`	L2	coda → `read(fd)` processo	VM (QEMU)
`vlan`	L2	aggiunge tag 802.1Q → padre	segmentazione
`macvlan`	L2	selezione per MAC → padre	container LAN
`vxlan`	L2	incapsula in UDP → `eth0`	overlay cloud
`bond`	L2	seleziona NIC slave → invia	failover/LAG
`gre`	L3	incapsula in IP → `eth0`	tunnel aziend.

Il principio

Per il kernel sono tutti `net_device` identici. Routing, firewall, Netlink e socket funzionano allo stesso modo. La differenza sta solo nelle due righe di `ndo_start_xmit`.

veth pair — il cavo virtuale tra due namespace

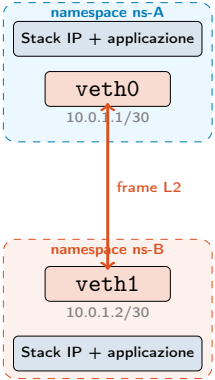
Come funziona

- Il kernel crea **due** `net_device` insieme
- Ognuno ha un puntatore all'altro: `priv->peer`
- `veth0.ndo_start_xmit(skb)`
  ⇒ `veth1->netif_rx(skb)`
- Appare in ricezione sull'altra estremità
- Frame Ethernet completi: MAC, ARP, VLAN
- Nessuna limitazione di banda reale (tutto in RAM)

Perchè servono i namespace

Senza namespace le due estremità sono nello stesso stack IP — non c'è isolamento reale.

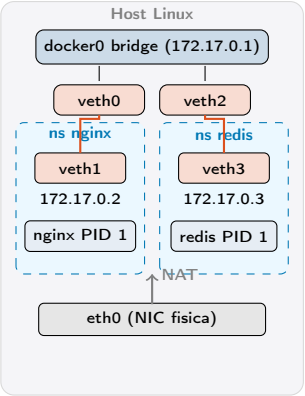
Spostare `veth1` in un altro namespace crea due stack IP completamente separati collegati da questo "cavo" virtuale.



veth pair e Docker — il quadro completo

Cosa fa Docker per ogni container

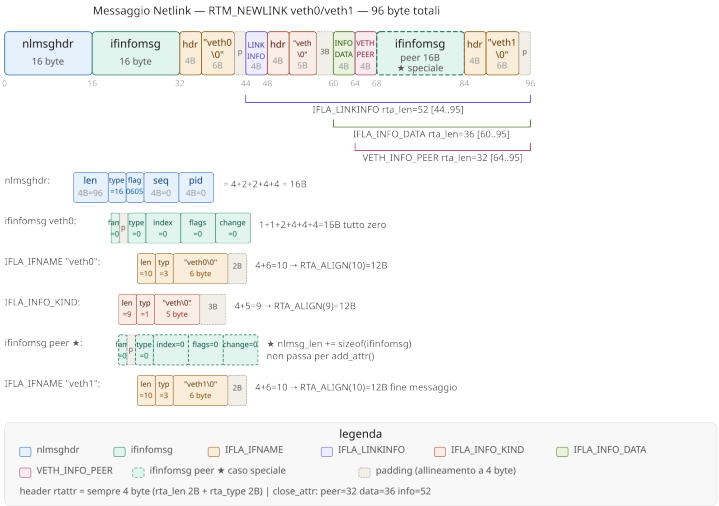
1. `clone(CLONE_NEWNET)` — nuovo namespace di rete
2. `RTM_NEWLINK type veth` — crea la coppia
3. `RTM_NEWLINK + IFLA_NET_NS_PID` — sposta `veth1` nel namespace del container
4. `RTM_NEWADDR` — assegna IP a `veth1`
5. `ip link set veth0 master docker0` — collega `veth0` al bridge `docker0`
6. `iptables` NAT — traffico verso internet



IFLA_NET_NS_PID

L'attributo Netlink che sposta un'interfaccia in un altro namespace. Aggiunto nel 2009 — impossibile con `ioctl/ifreq`. Questo è il motivo per cui esiste Netlink

Il messaggio veth



Il messaggio veth — struttura ad albero

Struttura ad albero degli attributi

```
nlmsghdr + ifinfomsg            [0..31]
IFLA_IFNAME = "veth0"           [32..43]
IFLA_LINKINFO                    [44..]
IFLA_INFO_KIND = "veth"
IFLA_INFO_DATA
VETH_INFO_PEER
ifinfomsg (vuoto)          caso speciale
IFLA_IFNAME = "veth1"

Totale:  96 byte
```

Confronto con dummy

	dummy	veth
Byte totali	60	96
INFO_DATA	no	sì
Peer	no	sì (annidato)
ifinfomsg extra	no	sì (caso speciale)

Perchè tanti livelli annidati?

- `IFLA_LINKINFO` raggruppa tutto ciò che riguarda il tipo
- `IFLA_INFO_DATA` contiene i parametri **specifici del tipo**
  (per veth: la descrizione del peer)
- `VETH_INFO_PEER` descrive la seconda interfaccia della coppia
- L'annidamento permette al parser del kernel di essere uniforme per tutti i tipi di interfaccia

Il caso speciale di VETH_INFO_PEER

`VETH_INFO_PEER` vuole un `ifinfomsg` vuoto **prima** degli attributi del peer. Va aggiunto a mano:

```
nlmsg_len += sizeof(ifinfomsg);
```

È l'unica eccezione al pattern standard di `add_attr()`.

crescita di nlmsg_len — veth vs dummy

dummy — 60 byte

Operazione	+byte	tot
`NLMSG_LENGTH(sizeof(ifi))`	32	32
`add_attr(IFLA_IFNAME,"dummy0",7)`	12	44
`add_attr(IFLA_LINKINFO,NULL,0)`	4	48
`add_attr(IFLA_INFO_KIND,"dummy",6)`	12	60
`close_attr(info)`	0	60

veth — 96 byte

Operazione	+byte	tot
`NLMSG_LENGTH(sizeof(ifi))`	32	32
`IFLA_IFNAME "veth0"`	12	44
`IFLA_LINKINFO`	4	48
`IFLA_INFO_KIND "veth"`	12	60
`IFLA_INFO_DATA`	4	64
`VETH_INFO_PEER`	4	68
`+= sizeof(ifinfomsg)`	16	84
`IFLA_IFNAME "veth1"`	12	96
`close_attr` $\times$ 3	0	96

close_attr in ordine inverso — perchè?

I tre close_attr con i valori reali

```
VETH_INFO_PEER inizia a offset 68
IFLA_INFO_DATA inizia a offset 64
IFLA_LINKINFO inizia a offset 44
fine messaggio = offset 96

close_attr(peer):
peer->rta_len = 96 - 68 = 28

close_attr(data):
data->rta_len = 96 - 64 = 32

close_attr(info):
info->rta_len = 96 - 44 = 52
```

La formula

`a->rta_len = &msg + nlmsg_len - a`

Misura la distanza tra la posizione finale del messaggio e l'inizio dell'attributo. Copre header + tutti i figli

Perchè ordine inverso?

Perchè tutti usano la stessa `nlmsg_len` finale (96).

L'ordine non è obbligatorio in questo caso specifico — ma è buona prassi chiudere dal figlio più interno al padre più esterno, come si chiude un tag HTML.

Verifica da shell

```
gcc -Wall -o veth_create veth_create.c
sudo ./veth_create
ip link show veth0
ip link show veth1
sudo ip link del veth0
# cancellare veth0 cancella anche veth1
```

Configurazione post-creazione da shell

Configurazione completa della veth pair con namespace

```
# 1.  Crea i namespace
ip netns add ns_A && ip netns add ns_B
# 2.  Crea la coppia (oppure usa sudo ./veth_create)
sudo ip link add veth0 type veth peer name veth1
# 3.  Sposta ciascuna estremita' nel namespace corrispondente
sudo ip link set veth0 netns ns_A
sudo ip link set veth1 netns ns_B
# 4.  Configura IP in ciascun namespace
sudo ip netns exec ns_A ip addr add 10.0.1.1/30 dev veth0
sudo ip netns exec ns_B ip addr add 10.0.1.2/30 dev veth1
# 5.  Porta UP
sudo ip netns exec ns_A ip link set veth0 up
sudo ip netns exec ns_A ip link set lo up
sudo ip netns exec ns_B ip link set veth1 up
sudo ip netns exec ns_B ip link set lo up
# 6.  Test
sudo ip netns exec ns_A ping -c 3 10.0.1.2
# 7.  Pulizia
sudo ip netns del ns_A && sudo ip netns del ns_B
```

In C: spostare veth1 in un namespace via Netlink

Aggiungere `IFLA_NET_NS_PID` al messaggio `RTM_NEWLINK` per `veth1`. Il kernel sposta l'interfaccia nel namespace del processo con quel PID. È ciò che fa Docker per ogni container.